

技术实现细节

创意洞察免费版报告的规则化模板拼接

为什么要做规则/半结构化模板填充和返回

1. **成本控制**：LLM按token计费，免费版用户量大将产生可观成本，且延迟增加（1-3秒），影响体验。
2. **价值区隔**：免费版只需提供“能看到特征，但不够聪明”的总结，付费版才提供归因和公式——这是转化付费的关键锚点。
3. **稳定性**：规则输出格式100%可控，不会出现幻觉或格式错乱，适合直接展示在UI上。

实现方案

- 用户对本次测试的某个胜出素材完成打标后，每个素材的标签结构应该通过JSON Schema的形式定义

```
{
  "material_id": "素材唯一标识",
  "test_id": "所属测试ID",
  "is_winner": true,
  "tags": {
    "hook": {
      "content_method": ["", ""],          // 最多2个，用户
      "elements": [""],                    // 最多2
      "emotion": [""]                      // 最多2个
    },
    "body": {
      "content_method": [""],              // 最多2个
      "elements": [""],                    // 最多2个
      "style": [""]                        // 最多2个
    },
    "cta": {
      "content_method": [""],              // 最多
      "elements": [""]
    }
  }
}
```

```

        "elements": [""], // 最多2个
    },
    },
    "custom_tags": { // 用户手动添加的标签
        "hook": {
            "content_method": [""],
            "elements": [],
            "emotion": []
        },
        "body": {
            "content_method": [],
            "elements": [""],
            "style": []
        },
        "cta": {
            "content_method": [],
            "elements": []
        }
    }
}

```

- 规则模板的创意特征总结填充：免费版不调用LLM，仅基于标签频次和预设模板生成自然语言描述
 - 如果只有一个打标完成的胜出素材，则直接罗列和填充相应的标签结果
 - 如果有 ≥ 2 个达标完成的胜出素材，需要对于每个category，选择出现次数最多的tag，（若并列第一且数量 $\leq$ 素材总数的一半，则标记为“多样”）

```

from typing import List, Dict, Any
from collections import defaultdict

# 预设标签的中文友好名称映射（用于模板）
LABEL_NAMES = {
    Pain Point Opening ":" Pain Point Opening "," Result
    Preceding ":" Result Preceding "," Curiosity Suspense
    ":" Curiosity Suspense ",
    Contrast/Surprise ":" Contrast Surprise "," Scene In

```

```

troduction ":" Scene Introduction "," Question Inquiry
":" Question Inquiry ",
    Discount/Price Pre positioning ":" Discount Price Pr
e positioning "," Brand Pre positioning ":" Brand Pre po
sitioning ",
    Headlines ":" Headlines "," Special Effects Subtitle
s ":" Special Effects Subtitles "," Sound Effects Enhanc
ement ":" Sound Effects Enhancement ",
    Sense of urgency: Sense of urgency, Sense of trust:
Sense of trust, Sense of pleasure: Sense of pleasure, Se
nse of impact: Sense of impact,
    Product Demonstration ":" Product Demonstration ","
Core Selling Point Explanation ":" Core Selling Point Ex
planation "," Differentiation ":" Differentiation ",
    Usage Scenario Display ":" Usage Scenario Display
"," Before and After Comparison ":" Before and After Com
parison "," Expert/Authoritative Endorsement ":" Expert
Endorsement ",
    Promotion Description ":" Promotion Description ","
Exclusive Information ":" Exclusive Information "," Pric
e ":" Price Information ",
    Promotion ":" Promotion Information "," Ingredients/
Specifications ":" Ingredient Specifications "," Subtitl
e Enhancement ":" Subtitle Enhancement ",
    Comment Screenshots ":" Comment Screenshots "," Prof
essionalism ":" Professionalism "," Authenticity ":" Aut
henticity ",
    Promotion ":" Promotion feeling "," Lifestyle ":" Li
festyle ",
    Button CTA ":" Button CTA "," Oral CTA ":" Oral CTA
"," Card CTA ":" Card CTA "," Brand Release ":" Brand Re
lease ",
    Buy Now ":" Buy Now "," Learn More ":" Learn More
"," Claim Discounts ":" Claim Discounts "," Add to Cart
":" Add to Cart "
}

def merge_tags(material_tags: Dict) -> Dict:

```

```

"""合并预设tags和custom_tags，返回完整标签字典"""
merged = {
    "hook": {"content_method": [], "elements": [],
"emotion": []},
    "body": {"content_method": [], "elements": [],
"style": []},
    "cta": {"content_method": [], "elements": []}
}
for section in ["hook", "body", "cta"]:
    for category in merged[section].keys():
        presets = material_tags["tags"].get(section,
{}).get(category, [])
        customs = material_tags.get("custom_tags",
{}).get(section, {}).get(category, [])
        merged[section][category] = presets + custom
s
    return merged

def generate_free_insight(winning_materials: List[Dict])
-> str:
    """
    输入：胜出素材列表，每个素材包含上面的tags结构
    输出：免费版创意特征总结文本
    """
    if not winning_materials:
        return "暂无胜出素材数据。"

    # 聚合所有胜出素材的标签
    agg = {
        "hook": {"content_method": defaultdict(int), "el
ements": defaultdict(int), "emotion": defaultdict(int)},
        "body": {"content_method": defaultdict(int), "el
ements": defaultdict(int), "style": defaultdict(int)},
        "cta": {"content_method": defaultdict(int), "ele
ments": defaultdict(int)}
    }

    for material in winning_materials:

```

```

merged = merge_tags(material)
for section in agg:
    for category in agg[section]:
        for tag in merged[section][category]:
            agg[section][category][tag] += 1

material_count = len(winning_materials)
lines = []

# 处理Hook部分
hook_lines = []
for cat, name in [("content_method", "内容呈现方法"),
("elements", "元素"), ("emotion", "情绪")]:
    if agg["hook"][cat]:
        # 找出频次最高的标签（如果并列且出现次数>1，都列出）
        max_freq = max(agg["hook"][cat].values())
        top_tags = [tag for tag, cnt in agg["hook"]
[cat].items() if cnt == max_freq]
        # 格式化标签名
        tag_names = [LABEL_NAMES.get(t, t) for t in
top_tags]
        if max_freq == material_count:
            hook_lines.append(f"{name}: 均采用「{'','.
join(tag_names)}」")
        else:
            hook_lines.append(f"{name}: 最常使用
「{'','.join(tag_names)}」（{max_freq}/{material_count}个素
材）")
    else:
        hook_lines.append(f"{name}: 无明确标注")
lines.append("【开场 Hook】\n" + "\n".join(f"  - {lin
e}" for line in hook_lines))

# 处理Body部分
body_lines = []
for cat, name in [("content_method", "内容呈现方法"),
("elements", "元素"), ("style", "风格")]:
    if agg["body"][cat]:

```

```

        max_freq = max(agg["body"][cat].values())
        top_tags = [tag for tag, cnt in agg["body"]
[cat].items() if cnt == max_freq]
        tag_names = [LABEL_NAMES.get(t, t) for t in
top_tags]

        if max_freq == material_count:
            body_lines.append(f"{name}: 均采用「{'','.
join(tag_names)}」")
        else:
            body_lines.append(f"{name}: 最常使用
「{'','.join(tag_names)}」({max_freq}/{material_count}个素
材)")
    else:
        body_lines.append(f"{name}: 无明确标注")
    lines.append("\n【主体 Body】\n" + "\n".join(f" - {l
ine}" for line in body_lines))

# 处理CTA部分
cta_lines = []
for cat, name in [("content_method", "引导方式"), ("e
lements", "呈现元素")]:
    if agg["cta"][cat]:
        max_freq = max(agg["cta"][cat].values())
        top_tags = [tag for tag, cnt in agg["cta"][c
at].items() if cnt == max_freq]
        tag_names = [LABEL_NAMES.get(t, t) for t in
top_tags]

        if max_freq == material_count:
            cta_lines.append(f"{name}: 均采用「{'','.j
oin(tag_names)}」")
        else:
            cta_lines.append(f"{name}: 最常使用「{'','.
join(tag_names)}」({max_freq}/{material_count}个素材)")
        else:
            cta_lines.append(f"{name}: 无明确标注")
    lines.append("\n【行动呼吁 CTA】\n" + "\n".join(f" -
{line}" for line in cta_lines))

```

```
return "\n".join(lines)
```

创意洞察付费版报告的LLM调用prompt

测试结果数据 + 素材创意标签数据的JSON

```
def build_llm_metadata(test_context: Dict, winning_materials: List[Dict]) -> Dict:
    """
    test_context: {
        "test_id": str,
        "metric": "点击率",
        "control_value": float,
        "treatment_value": float,
        "lift_percent": float,
        "p_value": float,
        "confidence": float,
        "sample_size": int
    }
    winning_materials: 胜出素材列表（每个包含完整tags）
    """
    merged_materials = [merge_tags(m) for m in winning_materials]
    material_count = len(merged_materials)

    # 统计每个category下的标签频次
    category_frequency = {
        "hook_content_method": defaultdict(int),
        "hook_elements": defaultdict(int),
        "hook_emotion": defaultdict(int),
        "body_content_method": defaultdict(int),
        "body_elements": defaultdict(int),
        "body_style": defaultdict(int),
        "cta_content_method": defaultdict(int),
        "cta_elements": defaultdict(int)
    }
```

```

for m in merged_materials:
    for tag in m["hook"]["content_method"]:
        category_frequency["hook_content_method"][tag]
+= 1
    for tag in m["hook"]["elements"]:
        category_frequency["hook_elements"][tag] += 1
    for tag in m["hook"]["emotion"]:
        category_frequency["hook_emotion"][tag] += 1
    for tag in m["body"]["content_method"]:
        category_frequency["body_content_method"][tag]
+= 1
    for tag in m["body"]["elements"]:
        category_frequency["body_elements"][tag] += 1
    for tag in m["body"]["style"]:
        category_frequency["body_style"][tag] += 1
    for tag in m["cta"]["content_method"]:
        category_frequency["cta_content_method"][tag] +
= 1
    for tag in m["cta"]["elements"]:
        category_frequency["cta_elements"][tag] += 1

# 转换为可读格式: {标签: 出现次数}
freq_readable = {k: dict(v) for k, v in category_freque
ncy.items()}

# 识别共性标签 (所有素材都有的标签)
common_tags = {}
for section_cat, freq_dict in category_frequency.items
():
    common = [tag for tag, cnt in freq_dict.items() if
cnt == material_count]
    if common:
        common_tags[section_cat] = common

# 识别差异标签 (只有部分素材有, 且至少有一个素材没有)
diff_tags = {}
for section_cat, freq_dict in category_frequency.items

```



```

():
    diff = [tag for tag, cnt in freq_dict.items() if 0
< cnt < material_count]
    if diff:
        diff_tags[section_cat] = diff

# 最终metadata
metadata = {
    "test_context": test_context,
    "winning_materials_summary": {
        "material_count": material_count,
        "category_frequency": freq_readable,
        "common_tags_across_all_materials": common_tag
s,
        "differential_tags": diff_tags
    }
}
return metadata

```

Prompt调用示例

```

import json

def call_llm_for_insight(metadata: Dict) -> str:
    system_prompt = """
        You are an expert in e-commerce short video creative in
sights. Generate a professional paid insight report based o
n user provided AB test data and tag statistics of winning
materials.

        The report must include the following three parts and b
e output in Markdown format (do not output JSON):
        ###Common summary
        Summarize the common creative features of all winning m
aterials on Hook/Body/CTA, using fluent Chinese paragraphs.
        ###Attribution Insights
        Based on the magnitude and significance of the improvem
ent in testing indicators, analyze which creative features

```

are most likely to be the key reasons for the improvement in effectiveness, and explain why (for example, a sense of urgency increases click desire, and before and after comparison enhances trust).

###Creative Formula

Extract a concise and reusable creative formula in the following format: 'Hook: {Content Method}+{Element}+{Emotion} → Body: {Content Method}+{Element} → CTA: {Guidance Method}+{Element}'

"""

```
user_prompt = f"以下是本次AB测试的数据和胜出素材的创意标签统计:\n```json\n{json.dumps(metadata, indent=2, ensure_ascii=False)}\n```"
```

```
# 此处调用OpenAI或其它LLM API
```

```
# response = openai.ChatCompletion.create(...)
```

```
# return response.choices[0].message.content
```

```
return "（实际调用LLM后的文本）"
```